
PHY3075 — Modélisation numérique en physique

Nom : Danny Thou
Matricule : 20278007

Date de remise : 26 mars 2026, 23h30
Devoir 4

1 Identification du projet

Mon projet consiste à créer et à entraîner un réseau de neurones pour identifier la présence d'exoplanète en utilisant un mélange de données publiques du Transiting Exoplanet Survey Satellite (TESS) et de Kepler.

2 Références

Les références ci-dessous sont les articles qui me serviront lors de la création de mon projet :

1. Álvarez, S. I., Alonso, E. D., Sánchez, M. L., Rodríguez, J. R., Lasheras, F. S., & Juez, F. J. de C. (2023). *One-dimensional Convolutional Neural Networks for Detecting Transiting Exoplanets* (arXiv :2312.07161). arXiv. <https://doi.org/10.48550/arXiv.2312.07161>
2. Wang, J. (2025). *Training a convolutional neural network for exoplanet classification with transit photometry data*. *Scientific Reports* 15(1), 15408. <https://doi.org/10.1038/s41598-025-98935-8>
3. ChamCarthy, K. (2025). *Advanced Modeling for Exoplanet Detection and Characterization* (arXiv :2506.17665). arXiv. <https://doi.org/10.48550/arXiv.2506.17665>
4. Dataset Kaggle, *Kepler Labelled Time Series Data*. <https://www.kaggle.com/datasets/keplersmachines/kepler-labelled-time-series-data/data?select=exoTrain.csv>
5. Fiscare, S., Ciaramella, A., Inno, L., Covone, G., Ferone, A., Rotundi, A., Hoffman, K. L., Quintana, E., Rowe, J. F., Bifulco, I., Cacciapuoti, L., Gallo, F., & Ienco, R. (2021). *Exploiting Kepler's Heritage : A Transfer Learning Approach for Identifying Exoplanets' Transits in TESS Data*. *Research Notes of the AAS*, 5(4), 91. <https://doi.org/10.3847/2515-5172/abf56b>
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. The MIT press. <http://www.deeplearningbook.org>

Une chose à noter est que le dataset permet de télécharger une série de données d'entraînement et de validation du télescope Kepler. Après avoir entraîné et validé le système, j'utiliserai de vraies données du télescope TESS pour identifier des courbes de flux dont on voit un transit d'exoplanète.

3 Méthode de modélisation

3.1 Réseau de neurones

Le modèle utilisé dans le cadre de ce projet sera un réseau de neurones convolutif (CNN). Celui-ci a été choisi pour sa capacité à analyser efficacement des variations dans une série de données en 1D. Cette capacité d'analyse sera cruciale dans la détection d'exoplanètes, puisque leurs présences est caractérisé par une diminution du flux lumineux de l'étoile pouvant être associée à un transit.

Avec les données du CNN, un perceptron multicouche (MLP) fera une classification binaire des courbes de flux selon la présence ou non d'un transit d'exoplanète.

Si le temps le permet, il serait intéressant de comparer les performances du MLP par rapport à d'autres modèles de classification. L'arbre de décision représenterait une bonne comparaison afin d'évaluer la pertinence de l'utilisation du MLP dans notre modèle.

3.2 Entraînement

Afin d'entraîner le modèle, l'optimiseur Adam (Adaptative Moments Estimations) sera utilisé. Cet algorithme permet un ajustement individuel du taux d'apprentissages des paramètres en se basant sur les gradients des itérations antérieurs. Cette modulation du taux d'apprentissage permet une convergence efficace vers un minimum de la fonction de perte. Voici le fonctionnement de l'optimiseur Adam (Goodfellow et al., 2016) :

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0,1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

FIGURE 1 – Algorithmme Adam

La fonction de perte représente la divergence entre la prédiction du modèle (q) et le résultat réel (p) et elle est donnée par :

$$H(p, q) = -\frac{1}{N} \sum_{x \in \chi} p(x) * \log(q(x)), \quad (1)$$

où χ est l'ensemble des données qu'on traite pour une courbe de flux.

4 Problèmes et angles à traiter

Tout d'abord, le problème principal du projet est de savoir si un CNN est capable de déterminer la présence d'exoplanète grâce à des courbes de flux et à quel point le réseau de neurones est capable de classer avec précision les différentes courbes que nous lui présentons. À ce problème s'ajoutent plusieurs angles possibles.

4.1 Données bruitées

Le premier angle consiste à déterminer la viabilité du réseau à bien identifier la présence lors le signal est bruité. En effet, aucun télescope n'est capable de donner un signal parfaitement lisse et le modèle doit être capable d'analyser les courbes même s'il y a du bruit.

4.2 Influence des paramètres physiques des exoplanètes

Le deuxième angle concerne les paramètres physiques des exoplanètes. Étant donné que chaque exoplanète est différente, les courbes de données peuvent varier et la diminution de flux causée par le passage d'une exoplanète peut être très bas. Il est alors important pour le réseau de différencier le bruit avec un passage d'une exoplanète.

4.3 Faux positif

Le troisième et dernier angle consiste à déterminer la capacité du modèle à distinguer le passage d'une exoplanète à un faux positif, qui peut être causé par du bruit ou une éclipse d'une étoile binaire.

4.4 Prétraitement des données

Si le temps le permet, il serait pertinent d'évaluer l'effet d'un prétraitement des données lors de l'entraînement du réseau et de comparer l'efficacité d'un réseau entraîné sur des données "propre" par rapport à un réseau entraîné directement avec les données brutes.

5 Méthode d'analyse

Dans le cadre de ce projet, la librairie Pytorch sera employée dans la création du modèle et de son analyse. En effet, Pytorch est une librairie *open source* permettant de construire facilement des réseaux de neurones. De plus, Pytorch est compatible avec les cœurs cuda qui sont des coeurs présents dans les cartes graphiques de NVIDIA et qui permettent de les utiliser pour accélérer les calculs en programmation. Puisque je possède une carte graphique compatible avec Pytorch, je l'utiliserai pour créer mon modèle et accélérer l'entraînement de mon réseau de neurones.

5.1 Fonction de perte

Comme mentionné à la section 3.2, la fonction de perte représente l'écart entre la prédiction du modèle et le résultat réel et est donnée par :

$$H(p, q) = -\frac{1}{N} \sum_{x \in \mathcal{X}} p(x) * \log(q(x)), \quad (2)$$

où χ est l'ensemble des données, $p(x)$ est le résultat réel et $q(x)$ est la prédiction du modèle. Pour bien entraîner le modèle, il est courant d'utiliser des boucles d'entraînement et de réutiliser les données d'entraînement plusieurs fois. Ainsi, nous pouvons définir une époque comme une itération complète des données d'entraînement. Avec ces époques, il est alors possible de tracer l'évolution temporelle de la fonction de perte, ce qui nous permet de voir la tendance de la fonction de perte à chaque époque.

5.2 Matrice de confusion

La matrice de confusion est une matrice permettant de visualiser la performance d'un modèle. Dans le cas d'une classification binaire, la matrice de confusion est composée de 4 catégories : les vrais positifs (VP - Réel=Positif et Prédiction=Positif), les vrais négatifs (VN - Réel=Négatif et Prédiction=Négatif), les faux positifs (FP - Réel=Négatif et Prédiction=Positif) et les faux négatifs (FN - Réel=Positif et Prédiction=Négatif). Voici à quoi ressemble la matrice de confusion :

$$\begin{bmatrix} VP & FN \\ FP & VN \end{bmatrix}$$

Cette matrice est cruciale, puisque les éléments de la matrice sont utilisés dans la quantification de la performance de la classification faite par le modèle.

5.3 Exactitude

L'exactitude du modèle représente la fraction de transits bien catégorisés et est donnée par :

$$Exactitude = \frac{VP + VN}{VP + FN + FP + VN} \quad (3)$$

5.4 Précision

La précision représente la fraction de transits qui ont bien été identifiés parmi toutes les données que le modèle considère comme possédant un transit :

$$précision = \frac{VP}{VP + FP} \quad (4)$$

5.5 Rappel

Le rappel est la fraction des transits qui ont bien été classés parmi toutes les données qui contiennent un transit. Concrètement, le rappel mesure le nombre de transits que le réseau arrive à bien identifier :

$$rappel = \frac{VP}{VP + FN} \quad (5)$$

5.6 F-mesure

La f-mesure est une mesure qui s'assure que le modèle classe bien les transits (précision) et identifie un grand nombre de transits (rappel). Cette mesure est donnée par :

$$\text{F-mesure} = 2 \times \frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}} \in [0, 1] \quad (6)$$

Plus F-mesure tend vers 1, plus le modèle classe bien les éléments qu'il analyse.

6 Pseudo-code

6.1 Réseau de neurones

Entrée:

- Courbe de flux

Sortie:

- Présence de transit ou non
- Pourcentage de confiance

Début

On définit la classe qui contiendra le réseau et son activation

On définit les fonctions du réseau:

*CNN:

- Couche de convolution 1D (16 filtres, noyau de 4 et foulée de 2)
- Activation par ELU
- Couche de convolution 1D (32 filtres, noyau de 8 et foulée de 2)
- Activation par ELU
- Couche de convolution 1D (64 filtres, noyau de 12 et foulée de 2)
- Activation par ELU
- Couche de convolution 1D (16 filtres, noyau de 20 et foulée de 2)
- Activation par ELU
- Couche de convolution 1D (16 filtres, noyau de 20 et foulée de 2)
- Activation par ELU
- Couche de convolution 1D (16 filtres, noyau de 4 et foulée de 2)
- Activation par ELU

*On crée une fonction pour aplatir le tenseur

*Perceptron multicouche (MLP):

- Couche avec 128 neurones
- Activation par ELU
- On déconnecte 15% des neurones entre la couche précédente et la suivante

- Couche avec 128 neurones
- Activation par ELU
- Couche avec 2 neurones
- On récupère le maximum de probabilité avec la fonction softmax

On définit la séquence d'opération que va effectuer l'objet:

- On applique le CNN sur les données
- On aplatit le tenseur
- On passe le tenseur aplati dans le MLP
- On retourne le maximum de probabilité

Fin

6.2 Entraînement

Entrée:

- Courbe de flux d'entraînement
- Nombre d'époques
- Le taux d'apprentissage
- Nombre de sous-divisions des données (lot)
- Optimiseur

Sortie:

- Le modèle entraîné
- Les catégories prédites

Début

On crée de listes pour contenir l'exactitude, la précision, le rappel et la f-mesure

Pour chaque époque:

*Pour chaque lot:

- On récupère la courbe et son résultat réel
- On calcule la prédiction du modèle
- On calcule la perte avec la fonction de perte

- On réinitialise les gradients de l'algorithme Adam
- On fait une rétropropagation
- On met à jour les paramètres

#Optionnel

*À chaque 50 lots:

- On affiche la valeur de la perte et le nombre de données traitées

- *On calcule la matrice de confusion
- *On calcule l'exactitude, la précision, le rappel et la f-mesure
- *On ajoute ces valeurs dans leur liste respective

On affiche les graphiques de l'exactitude, de la précision, du rappel et de la f-mesure.

Fin